

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

**MODUL 10 – STATE, HOOKS (USESTATE, USEEFFECT), DAN
CONDITIONAL RENDERING**

Nama Mahasiswa : Maessa Andrea Vallenia
NIM : 23051430044
Kelas : A
Tanggal Praktikum : 26 April 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 10 (Pertemuan 10)
Topik Utama	State Management, Lifecycle Hooks, dan Rendering Bersyarat.
Sub-Topik	State, Hooks (useState, useEffect), Conditional Rendering, Form Input State, Event Handling, Dependency Array, dan Real-time Update pada React.
Durasi Praktikum	340 Menit
Tanggal Praktikum	26 April 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami apa itu State dan perbedaannya dengan Props.
2	Menguasai penggunaan `useState` untuk menyimpan data yang berubah-ubah.
3	Memahami `useEffect` untuk menangani side effects (seperti timer atau API call).
4	Mampu melakukan Conditional Rendering (menyembunyikan/menampilkan elemen berdasarkan kondisi).

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
3	Live Server	Versi 5.7.9	Menjalankan HTML dan memantau perubahan file
4	JavaScript (Built-in Browser)	ES6 (Default Browser Modern)	Bahasa pemrograman untuk logika dan perhitungan
5	CodeSnap	Versi 1.3.4	Dokumentasi kode program
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework
8	Console Browser (Developer Tools)	Default Browser	Melihat output program dan debugging
9	Bootstrap	Versi 5.3	Membantu pembuatan UI seperti button, card, dan alert pada panel control
10	File praktikum	Pertemuan sebelumnya	Digunakan sebagai dasar pengembangan panel kontrol
11	LocalStorage (Web Storage API)	Built-in Browser	Digunakan untuk menyimpan data produksi secara persisten
12	API JSONPlaceholder	Built-in Browser	Digunakan sebagai

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
			simulasi database eksternal (data karyawan dan laporan)
13	Fetch API	Built-in Browser	Digunakan untuk melakukan HTTP request (GET, POST, dll) ke server

D. DASAR TEORI

1. State merupakan data internal pada komponen React yang nilainya dapat berubah selama aplikasi berjalan. State berbeda dengan props karena props hanya digunakan untuk menerima data dari komponen lain dan bersifat read-only, sedangkan state dapat diubah oleh komponen itu sendiri. Ketika state berubah, React akan otomatis melakukan re-render agar tampilan sesuai dengan data terbaru.
2. Hook adalah fitur pada React yang memungkinkan functional component menggunakan fitur React seperti state dan lifecycle. Sebelum adanya hook, fitur seperti state dan lifecycle hanya dapat digunakan pada class component. Dengan hook, penulisan kode menjadi lebih sederhana, ringkas, dan mudah dipahami.
3. useState merupakan hook yang digunakan untuk membuat dan mengelola state pada komponen React. useState mengembalikan dua nilai, yaitu state saat ini dan fungsi untuk mengubah state tersebut. Contohnya adalah `const [jumlah, setJumlah] = useState(0)` yang berarti state jumlah memiliki nilai awal 0.
4. useEffect merupakan hook yang digunakan untuk menangani side effect, seperti menjalankan timer, mengambil data dari API, atau mengubah judul halaman browser. useEffect dapat dijalankan saat komponen pertama kali ditampilkan, saat state berubah, atau saat komponen dihapus. Penggunaan dependency array menentukan kapan useEffect akan dijalankan.
5. Conditional Rendering adalah teknik menampilkan elemen tertentu berdasarkan kondisi tertentu. Dalam React, conditional rendering biasanya menggunakan if, ternary operator, atau operator logika `&&`. Teknik ini

berguna untuk menampilkan pesan, tombol, atau status tertentu sesuai kondisi aplikasi.

6. Event Handling pada React digunakan untuk menangani aksi pengguna seperti klik tombol, input data, atau perubahan pilihan. Event pada React ditulis menggunakan camelCase seperti `onClick`, `onChange`, dan `onSubmit`. Event handling sangat penting agar aplikasi menjadi interaktif.
7. Form input pada React biasanya dikontrol menggunakan state. Setiap perubahan nilai pada input akan langsung memperbarui state, sehingga data yang ditampilkan di layar selalu sinkron dengan data yang dimasukkan pengguna.

Secara keseluruhan, state, hooks, dan conditional rendering membuat aplikasi React menjadi lebih dinamis, interaktif, dan mampu menyesuaikan tampilan berdasarkan perubahan data secara real-time.

D.1 Konsep Utama

Konsep utama pada pertemuan ini adalah penggunaan state untuk menyimpan data yang dapat berubah selama aplikasi berjalan. State berfungsi sebagai memori sementara pada komponen React. Ketika nilai state berubah, React akan otomatis memperbarui tampilan sehingga pengguna dapat langsung melihat perubahan tersebut.

Penggunaan `useState` menjadi inti utama dalam praktikum karena hook ini digunakan untuk menyimpan nilai seperti jumlah produksi, target produksi, status mesin, dan input pengguna. Dengan `useState`, aplikasi dapat menjadi lebih interaktif karena data yang berubah langsung tercermin pada tampilan.

Selain `useState`, praktikum ini juga menggunakan `useEffect` untuk menangani proses yang berjalan secara otomatis, seperti timer pada jam digital. `useEffect` memungkinkan program menjalankan fungsi tertentu ketika komponen pertama kali ditampilkan, ketika state berubah, atau ketika komponen dihapus.

Konsep dependency array pada `useEffect` juga menjadi bagian penting. Jika dependency array kosong `[]`, maka `useEffect` hanya berjalan sekali saat komponen pertama kali ditampilkan. Jika dependency array berisi state tertentu, maka `useEffect` akan dijalankan kembali setiap kali state tersebut berubah.

Conditional rendering digunakan untuk menampilkan pesan atau tampilan yang berbeda sesuai kondisi tertentu. Contohnya, ketika jumlah produksi sudah mencapai target, sistem akan menampilkan pesan “Target Tercapai”. Sebaliknya, jika target belum tercapai, sistem akan menampilkan pesan “Produksi Berjalan”.

Konsep event handling juga diterapkan melalui tombol dan input form. Tombol tambah produksi, reset shift, dan emergency stop semuanya memanfaatkan event handler untuk menjalankan fungsi tertentu saat pengguna melakukan aksi.

Dengan menggabungkan state, hooks, event handling, dan conditional rendering, aplikasi React dapat menjadi sistem yang dinamis dan mampu merespon perubahan data secara real-time.

D.2 Keterkaitan dengan Teknik Industri

Materi state dan hooks memiliki hubungan yang erat dengan bidang Teknik Industri karena sistem industri modern memerlukan aplikasi yang mampu menampilkan data secara dinamis dan real-time. Dalam lingkungan industri, data produksi, kondisi mesin, jumlah output, dan status operator dapat berubah setiap saat.

Penggunaan state pada React sangat relevan untuk menampilkan data yang berubah-ubah, seperti jumlah produksi harian, target produksi, tingkat efisiensi mesin, dan jumlah barang cacat. Ketika data berubah, tampilan dashboard dapat langsung diperbarui tanpa perlu me-refresh halaman.

useEffect juga memiliki peran penting dalam sistem industri karena banyak proses memerlukan pembaruan otomatis. Contohnya adalah jam digital pada dashboard produksi, pembacaan sensor suhu setiap beberapa detik, atau pengambilan data mesin secara berkala dari server.

Conditional rendering sangat bermanfaat dalam sistem monitoring industri. Sebagai contoh, sistem dapat menampilkan pesan hijau ketika target produksi tercapai, pesan kuning ketika produksi masih berjalan, atau pesan merah ketika terjadi emergency stop. Hal ini mempermudah operator dalam memahami kondisi lapangan.

Konsep form input state juga dapat diterapkan pada sistem ERP dan quality control. Operator dapat memasukkan data jumlah produksi, waktu operasi mesin, jumlah produk cacat, dan berbagai data lainnya. Seluruh data tersebut kemudian diproses secara otomatis untuk menghasilkan informasi yang dibutuhkan.

Pada tugas proyek mini OEE, mahasiswa belajar menghitung Availability, Performance, dan Quality secara real-time. Perhitungan OEE sangat penting dalam Teknik Industri karena digunakan untuk mengukur efektivitas mesin dan proses produksi.

Dengan memahami state, hooks, dan conditional rendering, mahasiswa Teknik Industri dapat mengembangkan dashboard monitoring, sistem produksi, dan aplikasi industri yang lebih interaktif, responsif, dan mampu membantu pengambilan keputusan berbasis data.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Membuat Komponen Counter Produksi

Buat file baru `src/Komponen/CounterProduksi.jsx`.

Panggil komponen ini di `App.jsx`.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
```

1. JAWABAN KODE COUNTERPRODUKSI.JSX

```
import React, { useState } from 'react';  
function CounterProduksi() {
```

```

    // Deklarasi State: 'jumlah' bernilai awal 0, 'setJumlah' fungsi untuk
mengubahnya
    const [jumlah, setJumlah] = useState(0);
    const [target, setTarget] = useState(100);

    // Fungsi menambah produksi
    const tambahProduksi = () => {
        setJumlah(jumlah + 1);
    };

    // Fungsi reset
    const reset = () => {
        setJumlah(0);
    };

    return (
        <div className="text-center p-4 border rounded bg-light">
            <h3>Simulasi Hitung Produk</h3>
            <h1 className="display-4">{jumlah}</h1>
            <p>Target: {target} Unit</p>

            {/* Conditional Rendering: Tampilkan pesan jika target tercapai
*/}

            {jumlah >= target ? (
                <div className="alert alert-success d-inline-block">Target
Tercapai!</div>
            ) : (
                <div className="alert alert-secondary d-inline-
block">Produksi Berjalan...</div>
            )}

            <div className="mt-3">
                <button className="btn btn-primary me-2"
onClick={tambahProduksi}>
                    +1 Unit (Sensor OK)
                </button>
                <button className="btn btn-danger" onClick={reset}>
                    Reset Shift
                </button>
            </div>
        </div>
    );
}
export default CounterProduksi;

```

2. JAWABAN KODE APP.JSX

```
// 1. IMPORT: Beritahu React di mana letak file CounterProduksi
import CounterProduksi from "../Komponen/CounterProduksi";

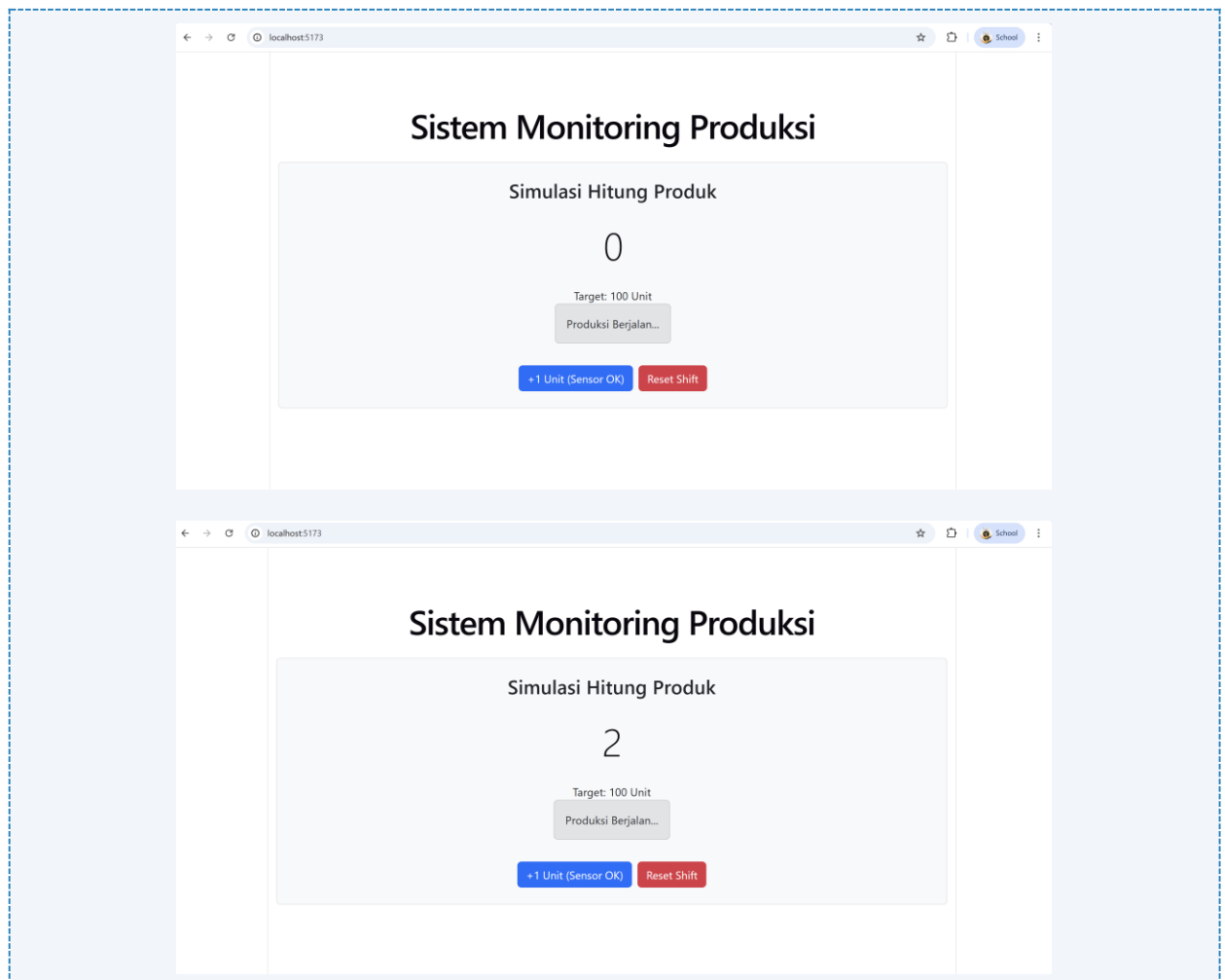
function App() {
  return (
    <div className="container mt-5">
      <h1 className="text-center mb-4">Sistem Monitoring Produksi</h1>

      {/* 2. PANGGIL: Tuliskan nama komponen seperti tag HTML */}
      <CounterProduksi/>

    </div>
  );
}

export default App;
```

Screenshot Hasil:



Gambar E.1 – Hasil Membuat Komponen Counter Produksi

E.2 Langkah 2 – Menggunakan useEffect untuk Real-time Clock

Kita akan membuat jam digital yang berjalan otomatis. Ini mensimulasikan pengambilan data live dari sensor tiap detik. Tambahkan kode di dalam `CounterProduksi.jsx` (atau buat komponen baru `JamDigital.jsx`):

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
```

1. JAWABAN KODE JAMDIGITAL.JSX

```
import React, { useState, useEffect } from 'react';
function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());

  // useEffect berjalan sekali setelah komponen dirender pertama kali
  useEffect(() => {

    // Membuat interval timer
    const timerID = setInterval(() => {
      setWaktu(new Date()); // Update state waktu setiap detik
    }, 1000);

    // Cleanup function: Dijalankan saat komponen dihapus/hancur
    // Penting untuk mencegah memory leak
    return () => {
      clearInterval(timerID);
    };
  }, []); // Array kosong [] artinya hanya dijalankan sekali saat mount

  return (
    <div className="alert alert-info text-center">
      <h4>Waktu Sistem Server: {waktu.toLocaleTimeString()}</h4>
    </div>
  );
}
export default JamDigital;
```

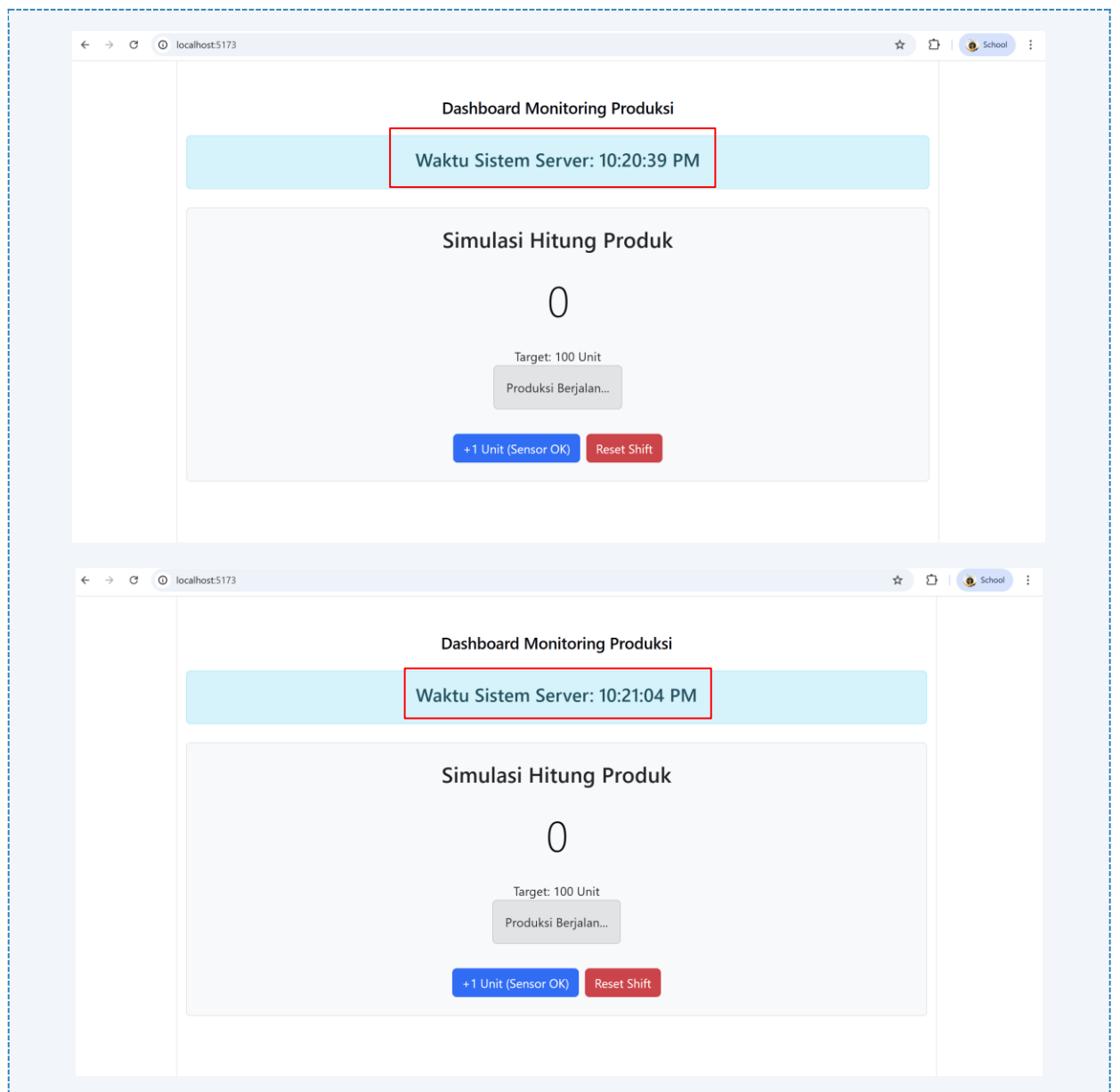
2. JAWABAN KODE APP.JSX

```
import CounterProduksi from "../Komponen/CounterProduksi";
import JamDigital from "../Komponen/JamDigital"; // Tambahkan ini

function App() {
  return (
    <div className="container mt-5">
      <h2 className="text-center mb-4">Dashboard Monitoring Produksi</h2>
```

```
{/* Jam Digital muncul di paling atas */}  
<JamDigital />  
  
<div className="mt-4">  
  <CounterProduksi />  
</div>  
</div>  
);  
}  
  
export default App;
```

Screenshot Hasil:



Gambar E.2 – Hasil Menggunakan useEffect untuk Real-time Clock

E.3 Langkah 3 – Contoh Form Input State

Ubah komponen `KartuMesin.jsx` dari pertemuan 9 agar statusnya bisa diedit secara lokal (opsional latihan):

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
```

1. JAWABAN KODE KARTUMESIN.JSX

```
import React, { useState } from 'react';

function KartuMesin({ nama, status, produksi }) {
  // State Lokal untuk ganti status (Langkah 3)
  const [statusLokal, setStatusLokal] = useState(status);

  return (
    <div className="card h-100 shadow-sm border-0 bg-white">
      <div className="card-body p-3 text-center">
        {/* Nama Mesin */}
        <h6 className="fw-bold text-primary mb-2">{nama}</h6>

        {/* Badge Status Dinamis */}
        <div className="mb-2">
          <span className={`badge ${
            statusLokal === 'Running' ? 'bg-success' :
            statusLokal === 'Stop' ? 'bg-danger' : 'bg-warning text-dark'
          }`}>
            {statusLokal}
          </span>
        </div>

        {/* Info Produksi */}
        <p className="small text-muted mb-3">
          Prod: <strong>{produksi}</strong> Unit</strong>
        </p>

        {/* Dropdown untuk ganti status */}
        <div className="mt-2">
          <select
            className="form-select form-select-sm"
            style={{ fontSize: '0.75rem' }}
            value={statusLokal}
            onChange={(e) => setStatusLokal(e.target.value)}
          >
            <option value="Running">Running</option>
            <option value="Stop">Stop</option>
            <option value="Maintenance">Maintenance</option>
          </select>
        </div>
      </div>
    </div>
  );
}
```

```

        </select>
      </div>
    </div>
  </div>
);
}

export default KartuMesin;

```

2. JAWABAN KODE APP.JSX

```

import React from 'react';
import CounterProduksi from './Komponen/CounterProduksi';
import JamDigital from './Komponen/JamDigital';
import KartuMesin from './Komponen/KartuMesin';

function App() {
  return (
    <div className="container mt-5">
      { /* Bagian Atas: Jam Digital */ }
      <div className="text-center mb-5">
        <JamDigital />
      </div>

      <div className="row g-4">
        { /* Sisi Kiri: Counter Produksi (Lebar 4) */ }
        <div className="col-lg-4">
          <div className="card shadow-sm p-4 h-100 border-0 bg-light">
            <CounterProduksi />
          </div>
        </div>

        { /* Sisi Kanan: Panel Kontrol Mesin (Lebar 8) */ }
        <div className="col-lg-8">
          <div className="card shadow-sm p-4 h-100 border-0">
            <h3 className="mb-4 text-center fw-bold">Panel Kontrol Mesin</h3>

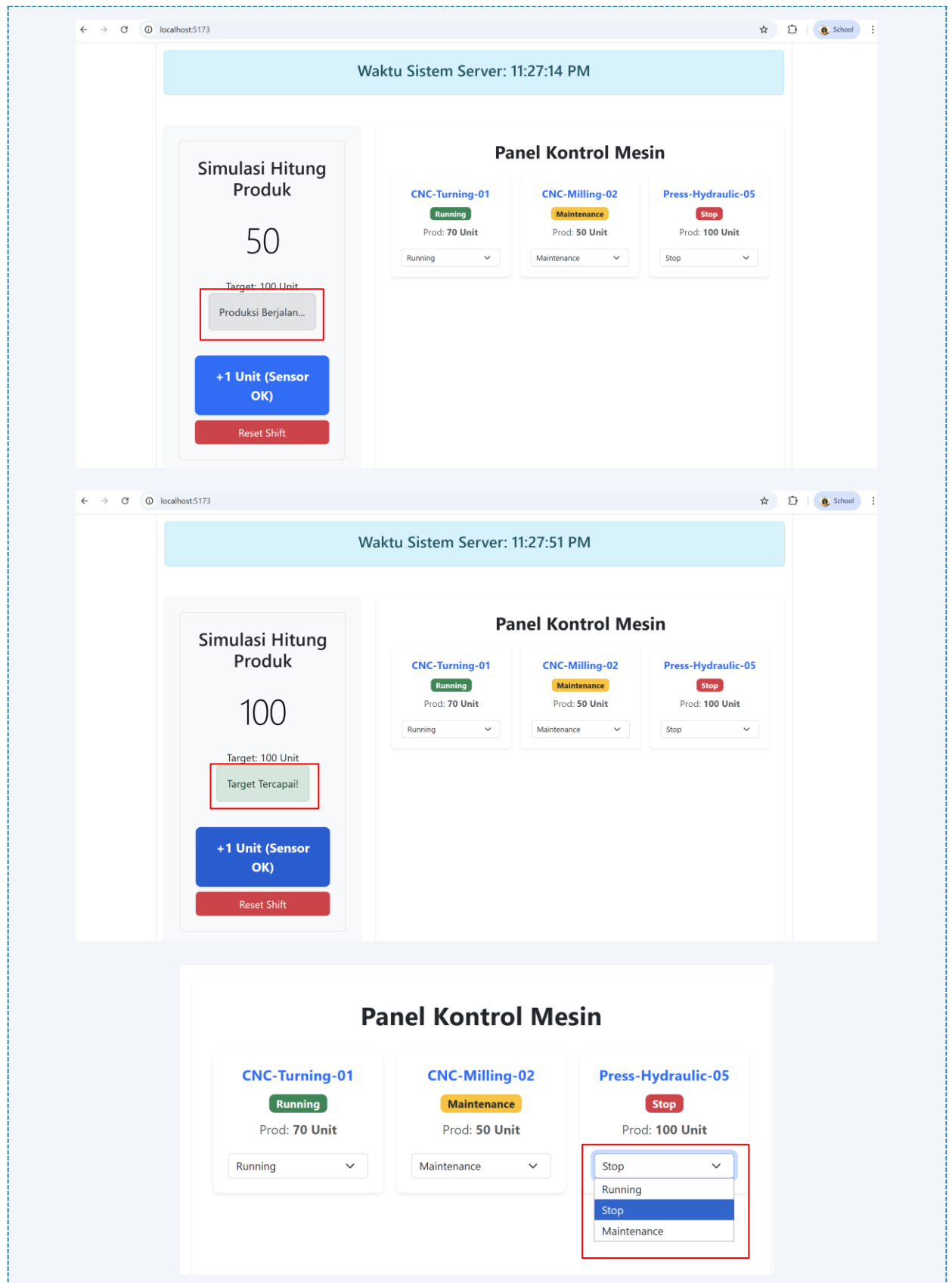
            { /* Kartu-kartu berjejer menyamping dengan row-cols-md-3 */ }
            <div className="row row-cols-1 row-cols-md-3 g-3">
              <div className="col">
                <KartuMesin nama="CNC-Turning-01" status="Running"
produksi={70} />
              </div>
              <div className="col">
                <KartuMesin nama="CNC-Milling-02" status="Maintenance"
produksi={50} />

```

```
        </div>
        <div className="col">
          <KartuMesin nama="Press-Hydraulic-05" status="Stop"
produksi={100} />
        </div>
      </div>
    </div>
  </div>
</div>
</div>
);
}

export default App;
```

Screenshot Hasil:



Gambar E.3 – Hasil Contoh Form Input State

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Dependency Array `useEffect`

Pertanyaan / Instruksi Latihan:

Modifikasi `JamDigital`. Tambahkan input text untuk memasukkan nama kota. Gunakan `useEffect` untuk mengubah `document.title` menjadi "Jam [Nama Kota]". Gunakan state kota sebagai dependency array `[kota]`.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';

function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());
  // 1. Tambahkan state untuk menyimpan nama kota
  const [kota, setKota] = useState('Jakarta');

  // useEffect untuk Update Jam (Setiap detik)
  useEffect(() => {
    const timer = setInterval(() => {
      setWaktu(new Date());
    }, 1000);
    return () => clearInterval(timer);
  }, []); // Array kosong karena jam jalan terus sejak awal

  // --- TUGAS MANDIRI 1: Update Document Title ---
  // useEffect ini hanya akan jalan kalau isi variabel [kota] berubah
  useEffect(() => {
    document.title = `Jam ${kota}`;
    console.log("Judul tab berubah!"); // Cek di console browser
  }, [kota]); // Inilah yang disebut Dependency Array

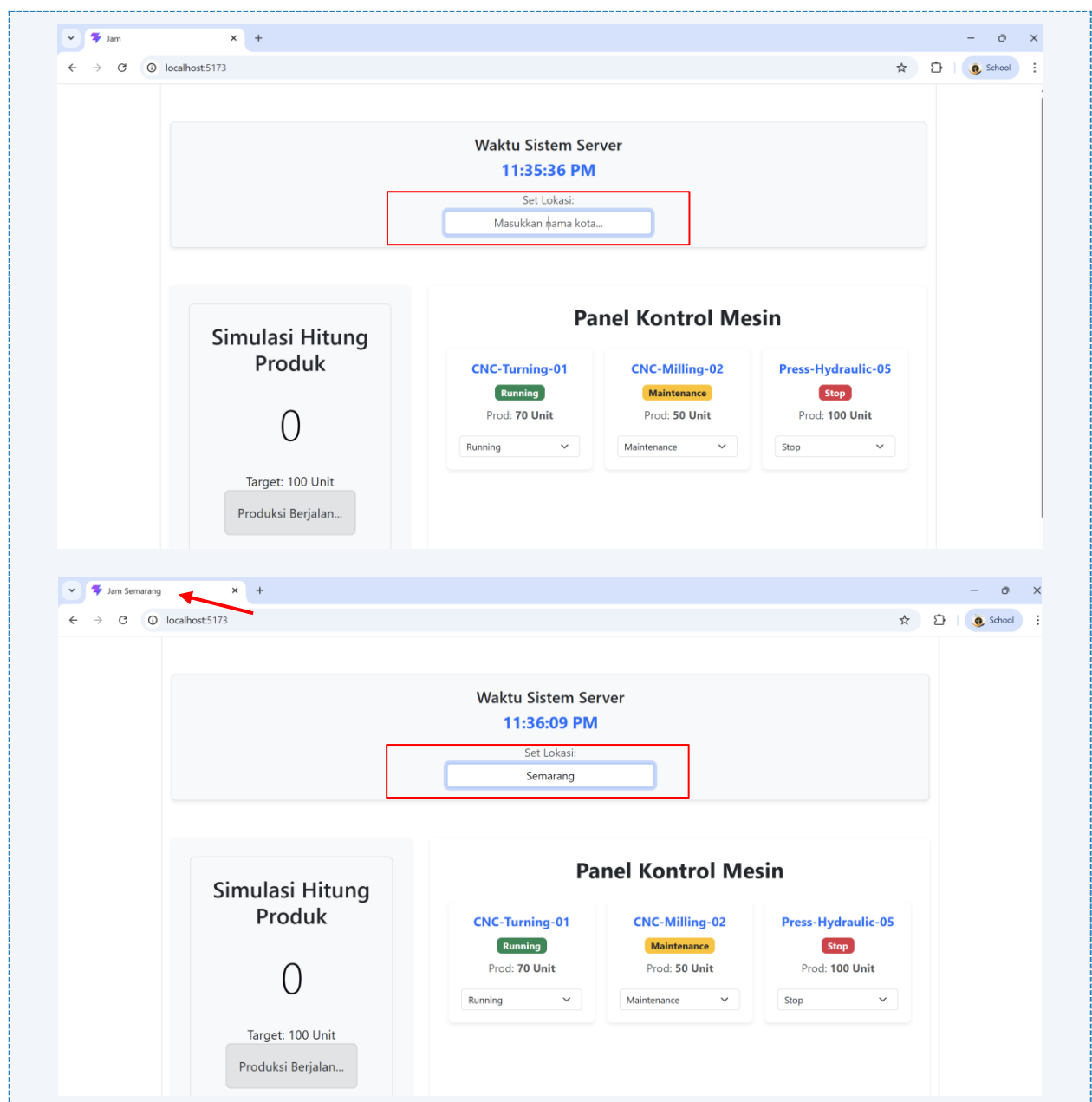
  return (
    <div className="text-center mb-4 p-3 border rounded bg-light shadow-sm">
      <h5>Waktu Sistem Server</h5>
      <h2 className="text-primary fw-bold">
        {waktu.toLocaleTimeString()}
      </h2>

      {/* Input Text untuk Nama Kota */}
      <div className="mt-3 mx-auto" style={{ maxWidth: '300px' }}>
        <label className="small d-block mb-1 text-muted">Set Lokasi:</label>
        <input
          type="text"

```

```
className="form-control form-control-sm text-center"
value={kota}
onChange={(e) => setKota(e.target.value)}
placeholder="Masukkan nama kota..."
/>
</div>
</div>
);
}

export default JamDigital;
```



Gambar F.1 – Hasil Dependency Array useEffect

F.2 Latihan 2 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Pada `CounterProduksi`, buat tombol "Emergency Stop". Saat diklik, ubah state status menjadi "EMERGENCY", tombol "+1" menjadi disabled (`disabled`), dan tampilkan pesan merah.

Jawaban / Kode Solusi:

```
import React, { useState } from 'react';

function CounterProduksi() {
  const [hitung, setHitung] = useState(0);
  const [isEmergency, setIsEmergency] = useState(false);

  return (
    <div className="text-center">
      <h4 className="text-muted mb-3">Simulasi Hitung Produk</h4>

      {/* Conditional Rendering Alert */}
      {isEmergency && (
        <div className="alert alert-danger fw-bold border-0 shadow-sm mb-4">
          STATE: EMERGENCY STOP ACTIVATED!
        </div>
      )}

      <h1 className={`display-1 fw-bold my-4 ${isEmergency ? 'text-danger' : 'text-dark'}`}>
        {hitung}
      </h1>

      <p className="text-secondary mb-4">Target: 100 Unit</p>

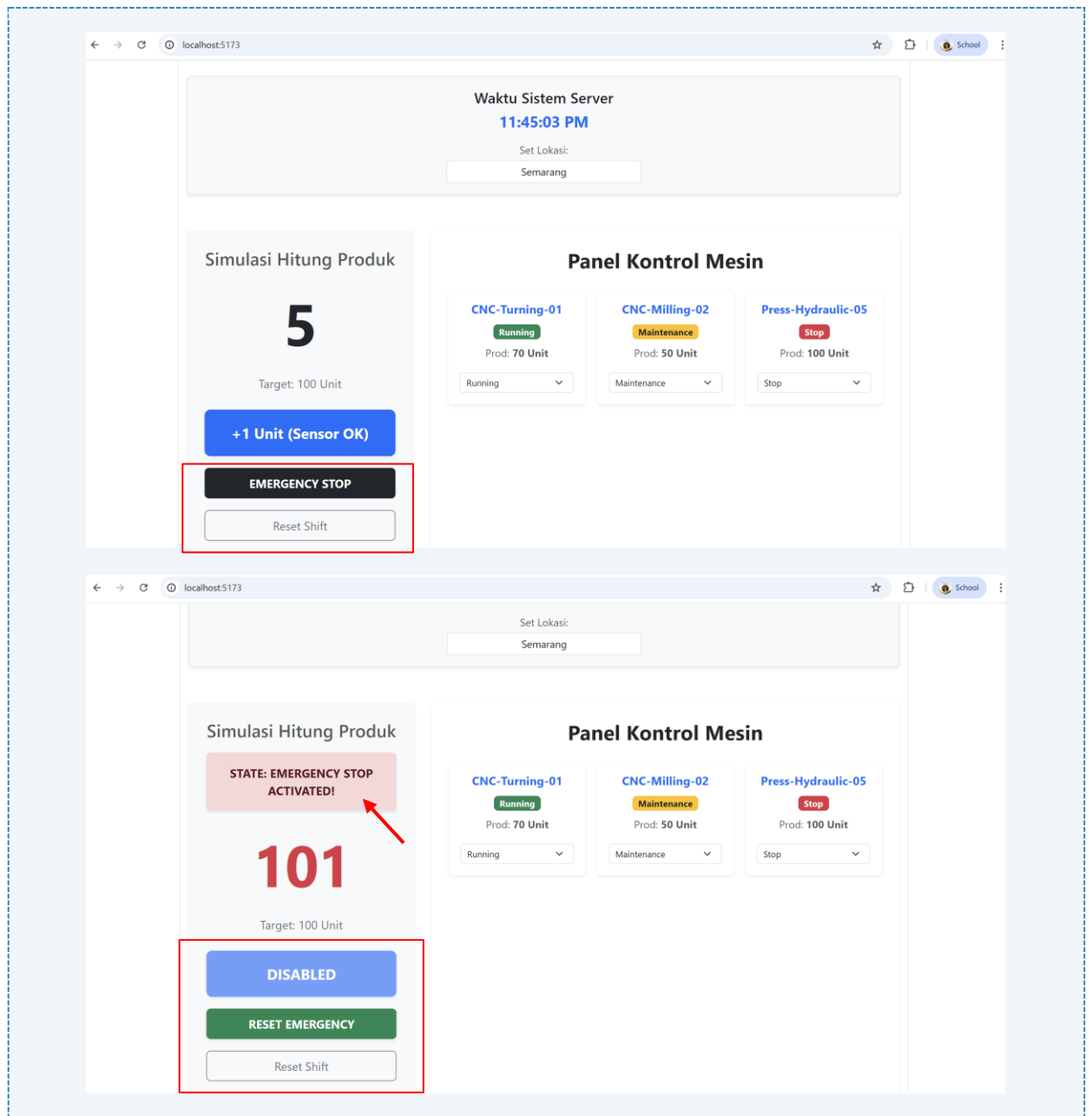
      <div className="d-grid gap-3">
        <button
          className="btn btn-primary btn-lg fw-bold py-3 shadow-sm"
          onClick={() => setHitung(hitung + 1)}
          disabled={isEmergency}
        >
          {isEmergency ? 'DISABLED' : '+1 Unit (Sensor OK)'}
        </button>

        {/* Tombol Emergency / Reset Normal */}
        {!isEmergency ? (
          <button
            className="btn btn-dark fw-bold py-2"
            onClick={() => setIsEmergency(true)}
          >
```

```
    >
      EMERGENCY STOP
    </button>
  ) : (
    <button
      className="btn btn-success fw-bold py-2 shadow-sm"
      onClick={() => setIsEmergency(false)}
    >
      RESET EMERGENCY
    </button>
  )}

  <button
    className="btn btn-outline-secondary py-2"
    onClick={() => {
      setHitung(0);
      setIsEmergency(false);
    }}
  >
    Reset Shift
  </button>
</div>
</div>
);
}

export default CounterProduksi;
```



Gambar F.2 – Hasil Conditional Rendering

F.3 Tugas Proyek Mini – Kalkulator OEE Sederhana

Deskripsi proyek mini:

- Buat komponen form input: 'Plan Time', 'Run Time', 'Total Parts', 'Good Parts'.
- Gunakan State untuk menyimpan nilai-nilai ini.
- Hitung OEE secara otomatis (Real-time) setiap kali input berubah.
- Rumus OEE = (Availability x Performance x Quality).
- Tampilkan hasil OEE dalam persen. Jika OEE < 50%, warnanya merah. Jika > 85%, warnanya hijau.

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini ini dilakukan melalui beberapa tahapan yang sistematis, dimulai dari perancangan tampilan hingga implementasi

1. Menyiapkan struktur komponen React menggunakan Vite, dengan mengimpor library pendukung seperti `useState` untuk manajemen data dan `useEffect` untuk sinkronisasi sistem.
2. Membuat state inputs untuk menampung empat variabel kunci produksi (Plan Time, Run Time, Total Parts, dan Good Parts) serta state history untuk menyimpan daftar riwayat perhitungan.
3. Membuat struktur tampilan menggunakan elemen input HTML yang dihubungkan dengan fungsi `handleChange`. Hal ini memastikan setiap perubahan angka yang diketik pengguna tersinkronisasi ke dalam state secara asinkron.
4. Mengimplementasikan rumus standar industri OEE ($\text{Availability} \times \text{Performance} \times \text{Quality}$). Perhitungan dilakukan secara langsung di dalam badan komponen agar hasil skor berubah seketika (*real-time*) setiap kali input diperbarui.
5. Menambahkan proteksi *Divide by Zero* (pembagian dengan nol) dan menggunakan `Math.min` untuk mengunci nilai maksimal pada angka 100%. Hal ini dilakukan untuk menjaga akurasi data agar tidak terjadi nilai yang tidak rasional (*infinity* atau *null*).
6. Membuat logika percabangan (*conditional rendering*) untuk menentukan status performa mesin berdasarkan ambang batas standar: Unacceptable (< 50%), Typical (50-85%), dan World Class (> 85%).
7. Mengintegrasikan class CSS Bootstrap secara kondisional agar warna teks, badge, dan progress bar berubah secara otomatis (Merah, Kuning, atau Hijau) sesuai dengan skor OEE yang dihasilkan.
8. Menambahkan sistem pendukung keputusan sederhana (*Decision Support System*) yang memberikan saran perbaikan teknis (seperti Kaizen atau TPM) berdasarkan status efisiensi yang terdeteksi.
9. Membuat fungsi `simpanKeRiwayat` untuk menangkap *snapshot* data saat itu (termasuk jam dan tanggal) dan menambahkannya ke dalam array riwayat menggunakan *spread operator*.
10. Menggunakan hook `useEffect` untuk menyimpan array riwayat ke dalam memori browser (*LocalStorage*). Fitur ini memastikan data laporan produksi tidak hilang meskipun halaman web dimuat ulang (*refresh*).
11. Menambahkan tombol "Reset Riwayat" yang dilengkapi dengan dialog konfirmasi (`window.confirm`) untuk menghapus seluruh data pada state dan membersihkan memori *LocalStorage* secara permanen.
12. Melakukan pengujian fungsionalitas dengan memasukkan berbagai skenario angka produksi, memverifikasi perubahan warna indikator, menguji ketahanan data saat *refresh*, dan memastikan fitur reset berjalan dengan benar.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';

function KalkulatorOEE() {
  const [inputs, setInputs] = useState({
```

```
    planTime: 480,
    runTime: 440,
    totalParts: 1000,
    goodParts: 950
  });

  // FITUR 1: Mengambil data dari localStorage saat pertama kali halaman
  dibuka
  const [history, setHistory] = useState(() => {
    const savedHistory = localStorage.getItem('oeo_history');
    return savedHistory ? JSON.parse(savedHistory) : [];
  });

  // FITUR 2: Menyimpan data ke localStorage setiap kali ada perubahan pada
  state history
  useEffect(() => {
    localStorage.setItem('oeo_history', JSON.stringify(history));
  }, [history]);

  const handleChange = (e) => {
    const { name, value } = e.target;
    const val = value === "" ? 0 : parseFloat(value);
    setInputs({ ...inputs, [name]: val });
  };

  // Logika Perhitungan OEE
  const availability = inputs.planTime > 0 ? Math.min(inputs.runTime /
  inputs.planTime, 1) : 0;
  const performance = inputs.runTime > 0 ? Math.min(inputs.totalParts /
  (inputs.runTime * 2.5), 1) : 0;
  const quality = inputs.totalParts > 0 ? Math.min(inputs.goodParts /
  inputs.totalParts, 1) : 0;
  const oee = (availability * performance * quality) * 100;

  // Penentuan Warna & Status
  let warnaTeks = "text-warning";
  let warnaBadge = "bg-warning text-dark";
  let statusLabel = "TYPICAL (Standar)";
  let rekomendasi = "Lakukan Preventive Maintenance untuk mengurangi minor
  stoppages.";

  if (oee < 50) {
    warnaTeks = "text-danger";
    warnaBadge = "bg-danger";
    statusLabel = "UNACCEPTABLE (Rendah)";
  }
}
```

```
    rekomendasi = "Efisiensi kritis! Segera lakukan Kaizen pada ketersediaan mesin.";
  } else if (oee > 85) {
    warnaTeks = "text-success";
    warnaBadge = "bg-success";
    statusLabel = "WORLD CLASS (Sempurna)";
    rekomendasi = "Performa luar biasa! Pertahankan ritme kerja lini produksi.";
  }

  const simpanKeRiwayat = () => {
    if (inputs.planTime === 0 || inputs.totalParts === 0) {
      alert("Plan Time dan Total Parts tidak boleh nol!");
      return;
    }
    const dataBaru = {
      id: Date.now(),
      jam: new Date().toLocaleString('id-ID'), // Menambahkan tanggal agar log lebih jelas
      hasil: oee.toFixed(1),
      ...inputs
    };
    setHistory([dataBaru, ...history]);
  };

  // FITUR 3: Fungsi Reset Riwayat
  const resetRiwayat = () => {
    if (window.confirm("Apakah Anda yakin ingin menghapus semua riwayat produksi?")) {
      setHistory([]);
      localStorage.removeItem('oee_history');
    }
  };

  return (
    <div className="container py-4">
      <div className="card shadow-lg border-0 rounded-4 overflow-hidden">
        <div className="bg-dark p-3 text-center">
          <h4 className="text-white fw-bold mb-0 text-uppercase">OEE Real-Time Monitor</h4>
          <small className="text-secondary text-uppercase">Maessa Andrea Vallenia - 23051430044</small>
        </div>

        <div className="card-body p-4 bg-light">
          <div className="row g-4">
```

```

<div className="col-lg-5">
  { /* INPUT PANEL */ }
  <div className="card border-0 shadow-sm p-3 mb-3 rounded-3
text-start">
    <h6 className="fw-bold text-muted mb-3"><i className="bi bi-
pencil-square me-2"></i>Data Produksi</h6>
    <div className="row g-2">
      <div className="col-6 mb-2">
        <label className="small fw-bold">Plan Time (Min)</label>
        <input type="number" name="planTime" className="form-
control" value={inputs.planTime} onChange={handleChange} />
      </div>
      <div className="col-6 mb-2">
        <label className="small fw-bold">Run Time (Min)</label>
        <input type="number" name="runTime" className="form-
control" value={inputs.runTime} onChange={handleChange} />
      </div>
      <div className="col-6 mb-2">
        <label className="small fw-bold">Total Parts</label>
        <input type="number" name="totalParts" className="form-
control" value={inputs.totalParts} onChange={handleChange} />
      </div>
      <div className="col-6 mb-2">
        <label className="small fw-bold">Good Parts</label>
        <input type="number" name="goodParts" className="form-
control" value={inputs.goodParts} onChange={handleChange} />
      </div>
    </div>
    <button className="btn btn-primary w-100 mt-3 fw-bold shadow-
sm" onClick={simpanKeRiwayat}>
      SIMPAN KE LOG
    </button>
  </div>

  { /* ANALYTICS RESULT */ }
  <div className="card border-0 shadow-sm p-4 text-center
rounded-3">
    <span className={`badge ${warnaBadge} mb-2 px-3 py-2 align-
self-center`} >{statusLabel}</span>
    <h1 className={`display-2 fw-bold ${warnaTeks} mb-
0`} >{oeo.toFixed(1)}%</h1>
    <div className="progress mt-3" style={{height: '12px'}}>
      <div className={`progress-bar progress-bar-striped
progress-bar-animated ${warnaBadge}`}
        style={{width: `${Math.min(oeo, 100)}%`} ></div>

```

```

        </div>
        <div className="alert alert-info mt-4 py-3 small border-0
shadow-sm text-start">
            <strong><i className="bi bi-lightbulb me-1"></i>
Rekomendasi Industri:</strong><br/>
            {rekomendasi}
        </div>
    </div>
</div>

<div className="col-lg-7">
    {/* LOG HISTORY */}
    <div className="card border-0 shadow-sm p-3 h-100 rounded-3
text-start">
        <div className="d-flex justify-content-between align-items-
center mb-3 border-bottom pb-2">
            <h6 className="fw-bold text-muted mb-0"><i className="bi
bi-clock-history me-2"></i>Riwayat Log Produksi</h6>
            {history.length > 0 && (
                <button className="btn btn-outline-danger btn-sm"
onClick={resetRiwayat}>
                    <i className="bi bi-trash me-1"></i> Reset
Riwayat
                </button>
            )}
        </div>

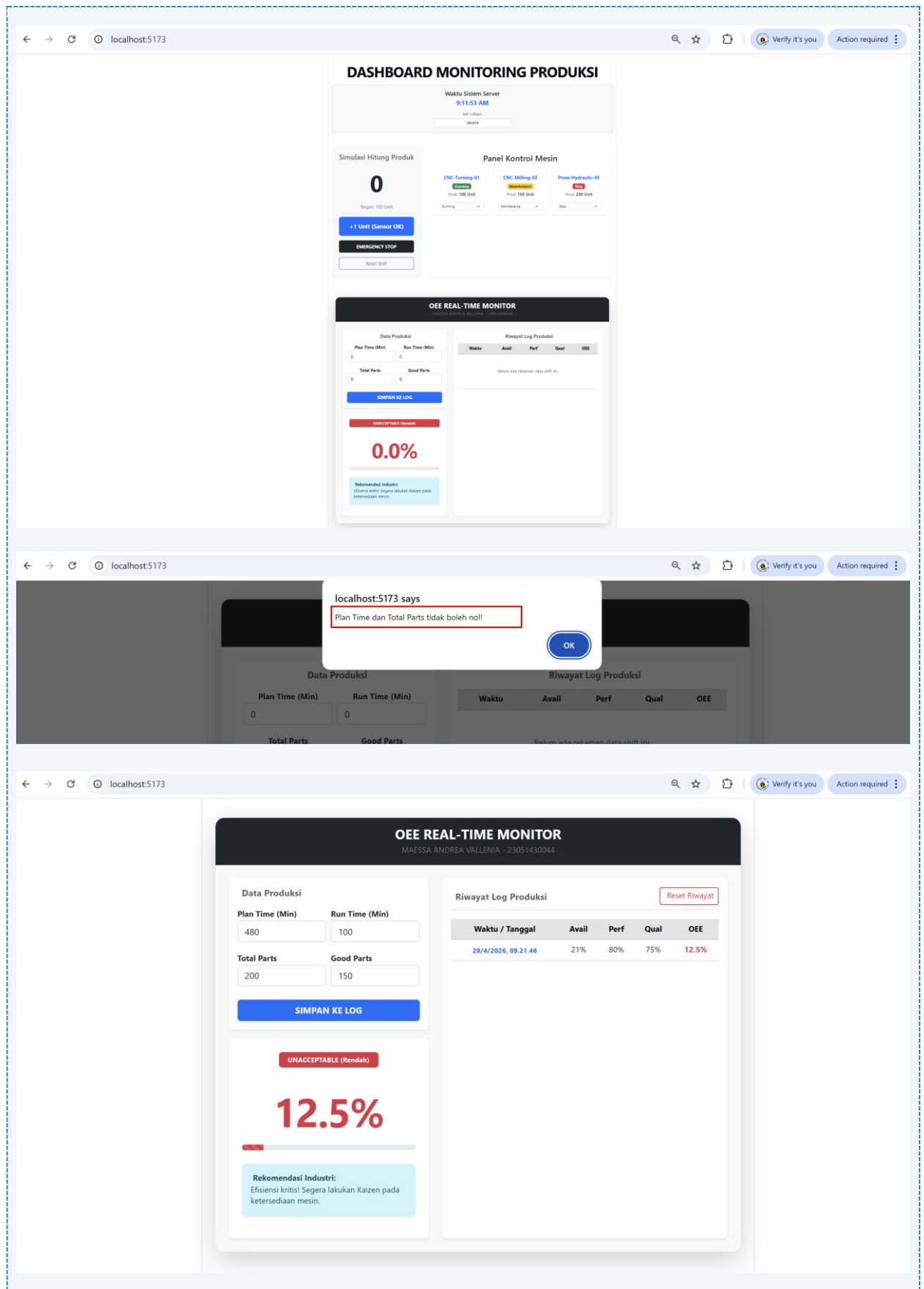
        <div className="table-responsive">
            <table className="table table-hover align-middle">
                <thead className="table-secondary">
                    <tr className="small text-center">
                        <th>Waktu / Tanggal</th>
                        <th>Avail</th>
                        <th>Perf</th>
                        <th>Qual</th>
                        <th>OEE</th>
                    </tr>
                </thead>
                <tbody className="small text-center">
                    {history.length === 0 ? (
                        <tr><td colspan="5" className="py-5 text-muted
italic">Belum ada rekaman data shift ini.</td></tr>
                    ) : (
                        history.map((log) => (
                            <tr key={log.id}>

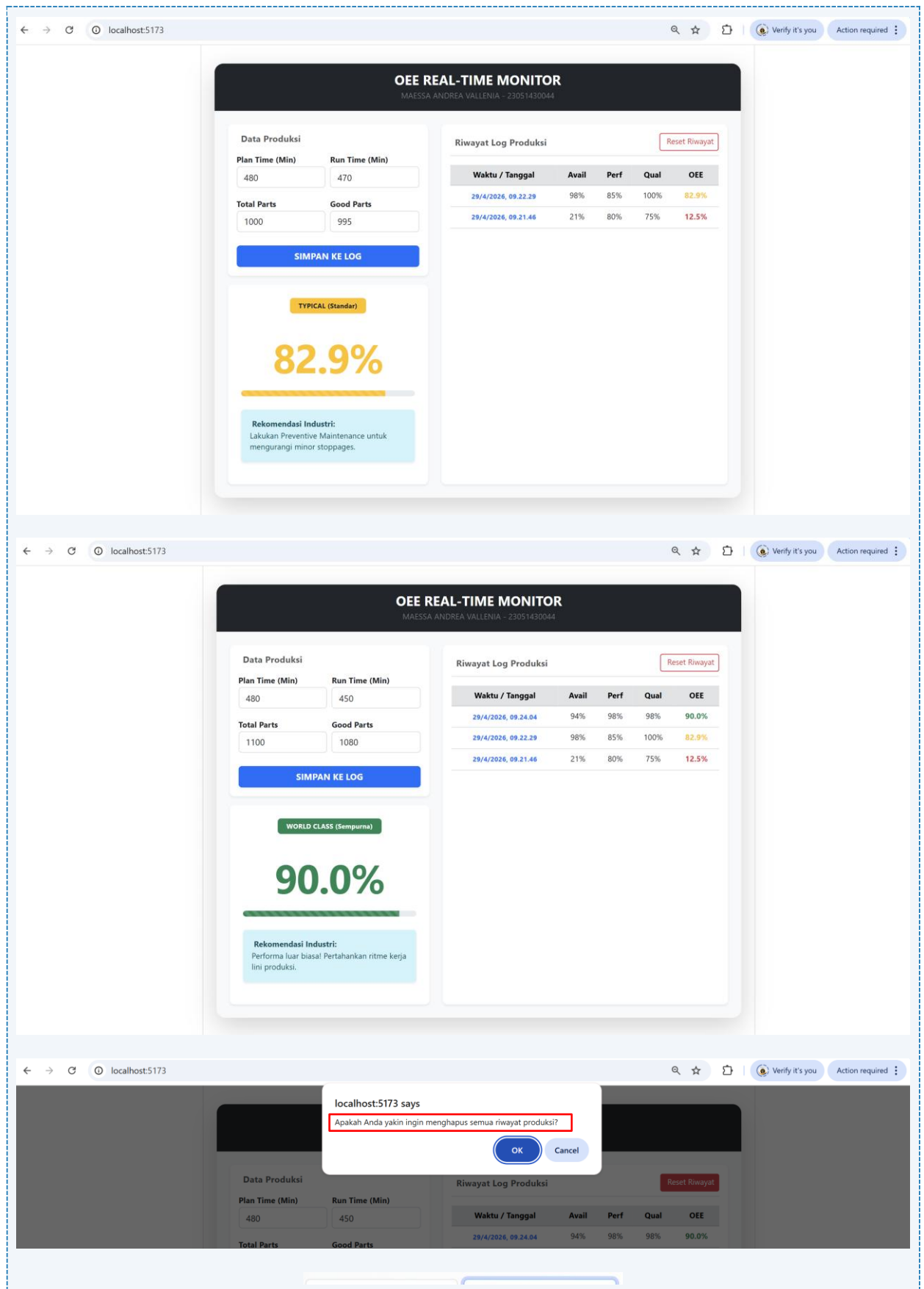
```



```
<td className="fw-bold text-primary"  
style={{fontSize: '0.75rem'}}>{log.jam}</td>  
      <td>(((log.runTime/log.planTime)*100).toFixed(0))  
%</td>  
  
      <td>(((log.totalParts/(log.runTime*2.5))*100).toF  
ixed(0))%</td>  
  
      <td>(((log.goodParts/log.totalParts)*100).toFixed  
(0))%</td>  
  
      <td className={`fw-bold ${log.hasil < 50 ? 'text-  
danger' : log.hasil > 85 ? 'text-success' : 'text-warning'}`}>  
        {log.hasil}%  
      </td>  
    </tr>  
  ))  
})  
</tbody>  
</table>  
</div>  
</div>  
</div>  
  
</div>  
</div>  
</div>  
</div>  
);  
}
```

```
export default KalkulatorOEE;
```





Gambar F.3 – Hasil Kalkulator OEE Sederhana

G. PEMBAHASAN

Praktikum Pertemuan 10 berfokus pada transformasi aplikasi React dari statis menjadi dinamis melalui penguasaan *State Management*, *Lifecycle Hooks*, dan *Conditional Rendering*. Pada tahap awal, pemahaman ditekankan pada konsep State sebagai memori internal komponen yang bersifat *read-write*, berbeda dengan *Props* yang bersifat *read-only*. Implementasi pertama dilakukan pada komponen CounterProduksi, di mana variabel jumlah dikelola menggunakan *hook* *useState*. Hal ini memungkinkan aplikasi untuk merespons interaksi pengguna secara instan, seperti penambahan unit produksi lewat sensor simulasi, yang secara otomatis memicu proses *re-render* untuk memperbarui tampilan angka di layar.

Selanjutnya, praktikum mengeksplorasi *Lifecycle Hooks* melalui penggunaan *useEffect* pada komponen JamDigital. Penggunaan *hook* ini sangat krusial dalam skenario industri untuk menangani *side effects* seperti sinkronisasi waktu sistem server secara *real-time*. Di sini, dipelajari pentingnya *Dependency Array* kosong `[]` agar timer hanya berjalan sekali saat komponen dimuat (*mount*), serta penggunaan *Cleanup Function* (`clearInterval`) untuk mencegah kebocoran memori (*memory leak*). Konsep ini memberikan simulasi bagaimana aplikasi dapat terus memperbarui data secara otomatis tanpa interaksi manual dari pengguna.

Pengembangan dilanjutkan dengan penerapan Form Input State pada komponen KartuMesin dan tugas mandiri. Mahasiswa melatih kemampuan dalam mengikat nilai input (*controlled component*) pada elemen select dan input text. Hal ini memungkinkan status mesin atau parameter judul halaman (`document.title`) berubah secara dinamis berdasarkan input yang dimasukkan. Pada tahap ini, pemahaman mengenai *asynchronous state update* mulai diperdalam, terutama saat menangani perubahan state yang saling bergantung satu sama lain.

Puncak praktikum ini adalah pembuatan Proyek Mini Kalkulator OEE. Proyek ini menggabungkan seluruh konsep yang telah dipelajari: penggunaan *useState* untuk menyimpan empat parameter utama produksi, penerapan logika matematis secara *real-time*, dan penggunaan *Conditional Rendering* untuk memberikan peringatan visual. Aplikasi ini mensimulasikan sistem pemantauan efisiensi industri yang cerdas, di mana antarmuka dapat berubah warna secara otomatis (Merah untuk efisiensi kritis $< 50\%$ dan Hijau untuk standar dunia $> 85\%$) sebagai bentuk umpan balik instan bagi operator produksi.

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum menunjukkan bahwa implementasi aplikasi telah sepenuhnya sesuai dengan teori State Management menggunakan React Hooks. Penggunaan `useState` pada variabel produksi dan parameter OEE terbukti efektif dalam menyimpan data yang bersifat dinamis. Sesuai dengan teori, setiap kali fungsi *setter* (seperti `setInputs`) dipanggil, React berhasil mendeteksi perubahan state dan melakukan pembaruan pada bagian DOM yang relevan tanpa mengganggu elemen lain, yang membuktikan efisiensi mekanisme *re-rendering* pada React.

Dalam aspek Lifecycle Hooks, penggunaan `useEffect` pada fitur jam digital dan sinkronisasi judul halaman telah sesuai dengan teori *side effects*. Hasil analisis menunjukkan bahwa penggunaan *dependency array* yang tepat sangat berpengaruh terhadap performa aplikasi. Sebagai contoh, pada latihan modifikasi judul dokumen, penggunaan `[kota]` dalam *dependency array* memastikan kode hanya berjalan ketika nama kota berubah, bukan pada setiap *render*. Hal ini selaras dengan teori optimasi performa dalam pengembangan perangkat lunak berbasis komponen.

Shutterstock

Analisis terhadap Conditional Rendering pada kalkulator OEE dan fitur *Emergency Stop* menunjukkan kesesuaian dengan prinsip *logic-driven UI*. Implementasi operator *ternary* atau logika AND (`&&`) di dalam JSX berhasil menampilkan pesan "Target Tercapai" atau mengubah status "EMERGENCY" secara akurat. Perubahan warna indikator (merah/hijau) berdasarkan nilai OEE membuktikan bahwa properti CSS dinamis dapat dikendalikan sepenuhnya oleh state, sesuai dengan teori yang menyatakan bahwa tampilan adalah fungsi dari data ($UI = f(state)$).

Terakhir, sinkronisasi data riwayat dengan `LocalStorage` (sebagai fitur tambahan) memperkuat teori tentang *persistence data* pada aplikasi *client-side*. Walaupun state bersifat sementara (akan hilang saat *refresh*), integrasi dengan `localStorage` memastikan data tetap bertahan. Secara keseluruhan, seluruh fitur yang dibangun mulai dari *counter* sederhana hingga kalkulator efisiensi industri telah memenuhi standar kompetensi praktikum dalam menciptakan aplikasi yang interaktif, memiliki alur logika yang kuat, dan mengikuti siklus hidup komponen yang benar.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Skor OEE menampilkan NaN atau Infinity saat kolom <i>Plan Time</i> atau <i>Total Parts</i> dikosongkan atau diisi angka 0.	Menambahkan validasi pengecekan kondisi (misal: <code>planTime > 0</code> ? hasil : 0) untuk memastikan pembagi tidak bernilai nol sebelum kalkulasi dilakukan.

G.3 Kaitan dengan Penerapan di Industri

Materi pada praktikum ini memiliki keterkaitan yang sangat erat dengan penerapan di dunia industri modern, khususnya dalam pengembangan Sistem Monitoring Produksi Terintegrasi, Smart Factory, dan Dashboard Efisiensi Mesin. Dalam industri manufaktur, data performa mesin tidak lagi dicatat secara manual, melainkan dipantau secara *real-time*. Konsep *State Management* yang dipelajari memungkinkan pembuatan dashboard yang interaktif, di mana setiap detik data dari sensor produksi dapat langsung ditampilkan dan dianalisis tanpa perlu memuat ulang halaman secara keseluruhan.

Penerapan Kalkulator OEE secara *real-time* adalah instrumen vital bagi manajemen pabrik. OEE (*Overall Equipment Effectiveness*) merupakan standar global untuk mengukur produktivitas manufaktur. Dengan sistem yang mampu menghitung otomatis faktor *Availability*, *Performance*, dan *Quality*, pihak manajemen dapat segera mengidentifikasi titik lemah produksi, seperti *downtime* yang tinggi atau tingkat *reject* yang melebihi ambang batas. Fitur Conditional Rendering (perubahan warna indikator) sangat relevan sebagai sistem peringatan dini (*Early Warning System*). Jika nilai OEE turun di bawah 50%, indikator merah akan segera memberi sinyal kepada supervisor untuk melakukan tindakan korektif atau *Emergency Stop*.

Penggunaan *Hooks* seperti *useEffect* untuk menjalankan timer atau sinkronisasi data menyerupai sistem SCADA (*Supervisory Control and Data Acquisition*) yang memantau ritme kerja mesin setiap waktu. Selain itu, penyimpanan data melalui *LocalStorage* memberikan gambaran kasar mengenai cara kerja database lokal pada perangkat IoT (*Edge Computing*) yang harus tetap menyimpan data meskipun koneksi internet terputus. Hal ini memastikan riwayat insiden dan performa mesin tetap terjaga untuk kebutuhan audit dan analisis jangka panjang.

Secara keseluruhan, pemahaman mendalam mengenai *State*, *Hooks*, dan *Conditional Rendering* membekali mahasiswa Teknik Industri dengan kemampuan untuk merancang solusi digital yang cerdas. Integrasi antara logika pemrograman dan prinsip manajemen industri ini sangat krusial dalam mendukung transformasi menuju Industri 4.0, di mana efisiensi operasional sangat bergantung pada ketersediaan data yang cepat, akurat, dan mudah dipahami oleh pengambil keputusan.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Mahasiswa mampu mengimplementasikan konsep <i>State Management</i> menggunakan hook <i>useState</i> untuk menyimpan dan memperbarui data parameter produksi secara dinamis tanpa memerlukan pemuatan ulang halaman (reload).
2	Mahasiswa dapat menggunakan hook <i>useEffect</i> untuk menangani <i>side effects</i> seperti pembuatan jam digital <i>real-time</i> dan sinkronisasi data ke <i>LocalStorage</i> , guna memastikan kontinuitas data riwayat produksi pada browser.

3	Mahasiswa berhasil menerapkan teknik Conditional Rendering untuk memberikan umpan balik visual instan, seperti perubahan warna indikator performa (Merah/Kuning/Hijau) berdasarkan logika ambang batas skor OEE.
4	Mahasiswa mampu membangun logika perhitungan matematis yang kompleks secara otomatis (<i>real-time</i>) dengan validasi keamanan input untuk mencegah error teknis seperti <i>Divide by Zero</i> dan nilai <i>out-of-bounds</i> .
5	Mahasiswa memahami cara mengintegrasikan interaksi pengguna (<i>event handling</i>) dengan logika industri, sehingga mampu menciptakan dashboard monitoring efisiensi mesin yang responsif dan sesuai dengan kebutuhan analisis performa di Industri 4.0.

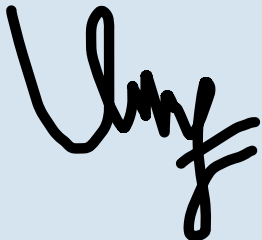
I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Kom, I. L. M. (2023). <i>Dasar Algoritma dan Pemrograman Javascript</i> . Irvan Lewenusa, M. Kom.
[3]	Sholikhan, M. (2022). Html, Css Dan Javascript. <i>Penerbit Yayasan Prima Agus Teknik</i> , 1-343 .
[4]	Torres, J. A. G., Lau, S. H., Anchuri, P., Stevens, J. M., Tabora, J. E., Li, J., ... & Doyle, A. G. (2022). A multi-objective active learning platform and web app for reaction optimization. <i>Journal of the American Chemical Society</i> , 144(43), 19999-20007.
[5]	Patil, K., & Javagal, S. D. (2022). React state management and side-effects—A Review of Hooks. <i>International Research Journal Of Engineering And Technology (IRJET)</i> , 9, 12.
[6]	https://developer.mozilla.org/en-US/docs/Web/JavaScript
[7]	https://www.geeksforgeeks.org/
[8]	https://react.dev/reference/react/useState
[9]	https://react.dev/
[10]	https://react.dev/reference/react/useState
[11]	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
[12]	https://getbootstrap.com/docs/5.3/getting-started/introduction/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Maessa Andrea Vallenia NIM: 23051430044	 Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____